

2. Supernumerary tentacles

Contents

1	Environment preparation	1
2	Supernumerary tentacles thresholds	2
3	Data formatting	2
4	Survival object	4
5	Proportional hazards assumption check : Schoenfeld residuals	4
6	Model selection	4
6.1	Random effects selection	4
6.2	Fixed effects selection	5
6.3	Goodness-of-fit evaluation	5
6.4	Selected model summary	6
7	Median times by tumoral state	7
8	Cumulative incidences by week and tumoral state	7
9	Cumulative proportions by week and tumoral state	8
10	Plots of cumulative incidence of supernumerary tentacles development by tumoral state	9

1 Environment preparation

Run scripts with the needed packages, functions, and the raw data frame.

```
source("0a_packages.R")
source("0b_script_data.R")
source("0c_functions.R")
```

2 Supernumerary tentacles thresholds

For each lineage, determine the supernumerary tentacle threshold, defined as the maximum tentacle number observed among healthy individuals kept in control sleep conditions.

```
df %>%
  filter(tum_state == "Healthy", sleep_condition == "Control") %>%
  pivot_longer(cols = starts_with("tenta_"),
               names_to = "time",
               values_to = "tentacles") %>%
  group_by(lineage) %>%
  summarise(max_tentacles = max(tentacles, na.rm = TRUE),
            .groups = "drop")
```

```
## # A tibble: 6 x 2
##   lineage max_tentacles
##   <fct>      <dbl>
## 1 HC_MT          8
## 2 HO_MT          7
## 3 HO_SPC         7
## 4 HO_SPT         7
## 5 HO_VLN         8
## 6 HV_GAL        11
```

3 Data formatting

Create a new data frame adapted for survival analysis.

```
tenta_df <-

  #' Load the raw data frame.
  df %>%

  #' Restrict the data frame to HO_SPT lineage.
  filter(lineage == "HO_SPT") %>%

  #' Define the supernumerary tentacles threshold specific to HO_SPT
  #' as calculated above.
  mutate(threshold = 7) %>%

  rowwise() %>%

  #' Combine the weekly tentacle counts (tenta_1 to tenta_6) into a
  #' single vector per individual.
  mutate(tenta_vec = list(c(tenta_1, tenta_2, tenta_3, tenta_4, tenta_5, tenta_6))) %>%

  #' Keep only individuals with at least one tentacle count.
  filter(any(!is.na(tenta_vec))) %>%

  #' For each individual, identify:
  #' - the first week at which the tentacle count exceeds the threshold (event),
  #' - the last week with a non-missing tentacle count (used for censoring),
```

```

# - a binary event: 1 if supernumerary tentacles occurred, 0 otherwise.
mutate(first_week_tenta_event = which(tenta_vec > threshold)[1],
       last_tenta_week = max(which(!is.na(tenta_vec))),
       tenta_event = if_else(!is.na(first_week_tenta_event), 1, 0)) %>%

ungroup() %>%

# Define the follow-up time for each individual:
# if the hydra developed supernumerary tentacles -> time of first occurrence (event),
# if the hydra died -> time of death (censor),
# if the hydra did not develop supernumerary tentacles and stayed alive
# -> last time with available data (no event, censor at the end of the experiment).
mutate(age_death_weeks = age_death / 7,
       age_followup = case_when(tenta_event == 1 ~ as.numeric(first_week_tenta_event),
                                !is.na(age_death_weeks) & tenta_event == 0 ~
                                age_death_weeks,
                                TRUE ~ as.numeric(last_tenta_week))) %>%

# Build the variable tum_state_early: an individual is considered
# tumoral only if the tumor appeared before or at the same time as the
# supernumerary tentacle event.
mutate(age_tum_weeks = age_tum / 7,
       tum_state_early = case_when(tum_state == "Tumoral" &
                                   !is.na(age_tum_weeks) &
                                   age_tum_weeks <= age_followup ~ "Tumoral",
                                   TRUE ~ "Healthy"),
       tum_state_early = factor(tum_state_early, levels = c("Healthy", "Tumoral"))) %>%

# Build a competing-risk variable for the cumulative incidence functions:
# if the hydra did not develop supernumerary tentacles and stayed alive -> 0 (no
# event),
# if the hydra developed supernumerary tentacles -> 1 (event),
# if the hydra died -> 2 (competing risk of death).
mutate(status_competing = factor(case_when(tenta_event == 1 ~ 1,
                                           !is.na(age_death_weeks) & tenta_event == 0 ~
                                           2,
                                           TRUE ~ 0))) %>%

# Keep only the variables needed for the analysis.
select(age_followup, tenta_event, status_competing, lineage, sleep_condition,
       tum_state_early,
       id_unique, parent_unique, replicate_unique) %>%

droplevels()

```

Set reference levels for the categorical variables.

```

tenta_df$sleep_condition <- relevel(tenta_df$sleep_condition, ref = "Control")
tenta_df$tum_state_early <- relevel(tenta_df$tum_state_early, ref = "Healthy")

```

4 Survival object

Create a survival object with follow-up time and event indicator.

```
tenta_surv_object <- Surv(tenta_df$age_followup, tenta_df$tenta_event)
```

5 Proportional hazards assumption check : Schoenfeld residuals

Fit a preliminary full Cox model (fixed effects only).

```
tenta_cox <- coxph(Surv(age_followup, tenta_event) ~ sleep_condition + tum_state_early,  
data = tenta_df)
```

Test the proportional hazards assumption.

```
print(cox.zph(tenta_cox))
```

```
##                chisq df    p  
## sleep_condition  2.37  1 0.12  
## tum_state_early  2.26  1 0.13  
## GLOBAL          4.23  2 0.12
```

The proportional hazards assumption is not violated for any term, a Cox mixed-effects model will be used.

6 Model selection

6.1 Random effects selection

With the full fixed-effect structure held constant, compare different random-effect structures to find the optimal one.

```
tenta_R_0 <- coxph(Surv(age_followup, tenta_event) ~ sleep_condition + tum_state_early,  
data = tenta_df)  
  
tenta_R_1 <- coxme(Surv(age_followup, tenta_event) ~ sleep_condition + tum_state_early +  
(1 | parent_unique), data = tenta_df)  
  
tenta_R_2 <- coxme(Surv(age_followup, tenta_event) ~ sleep_condition + tum_state_early +  
(1 | replicate_unique), data = tenta_df)  
  
tenta_R_3 <- coxme(Surv(age_followup, tenta_event) ~ sleep_condition + tum_state_early +  
(1 | parent_unique) + (1 | replicate_unique), data = tenta_df)  
  
AIC(tenta_R_0, tenta_R_1, tenta_R_2, tenta_R_3) %>% arrange(AIC)  
  
##                df      AIC  
## tenta_R_1 3.517649 817.2053  
## tenta_R_3 3.655228 817.2116  
## tenta_R_0 2.000000 817.3347  
## tenta_R_2 2.040074 817.3456
```

The model without any random effect (tenta_R_0) has the lowest AIC, this random effect structure is therefore selected. A standard Cox model (coxph) is thus used for the remaining steps.

6.2 Fixed effects selection

With the selected random-effect structure fixed, compare different fixed-effect structures to find the optimal one.

```
tenta_F_0 <- coxph(Surv(age_followup, tenta_event) ~ 1, data = tenta_df)
tenta_F_1 <- coxph(Surv(age_followup, tenta_event) ~ sleep_condition, data = tenta_df)
tenta_F_2 <- coxph(Surv(age_followup, tenta_event) ~ tum_state_early, data = tenta_df)
tenta_F_3 <- coxph(Surv(age_followup, tenta_event) ~ sleep_condition + tum_state_early,
data = tenta_df)
tenta_F_4 <- coxph(Surv(age_followup, tenta_event) ~ sleep_condition * tum_state_early,
data = tenta_df)

AIC(tenta_F_0, tenta_F_1, tenta_F_2, tenta_F_3, tenta_F_4) %>% arrange(AIC)
```

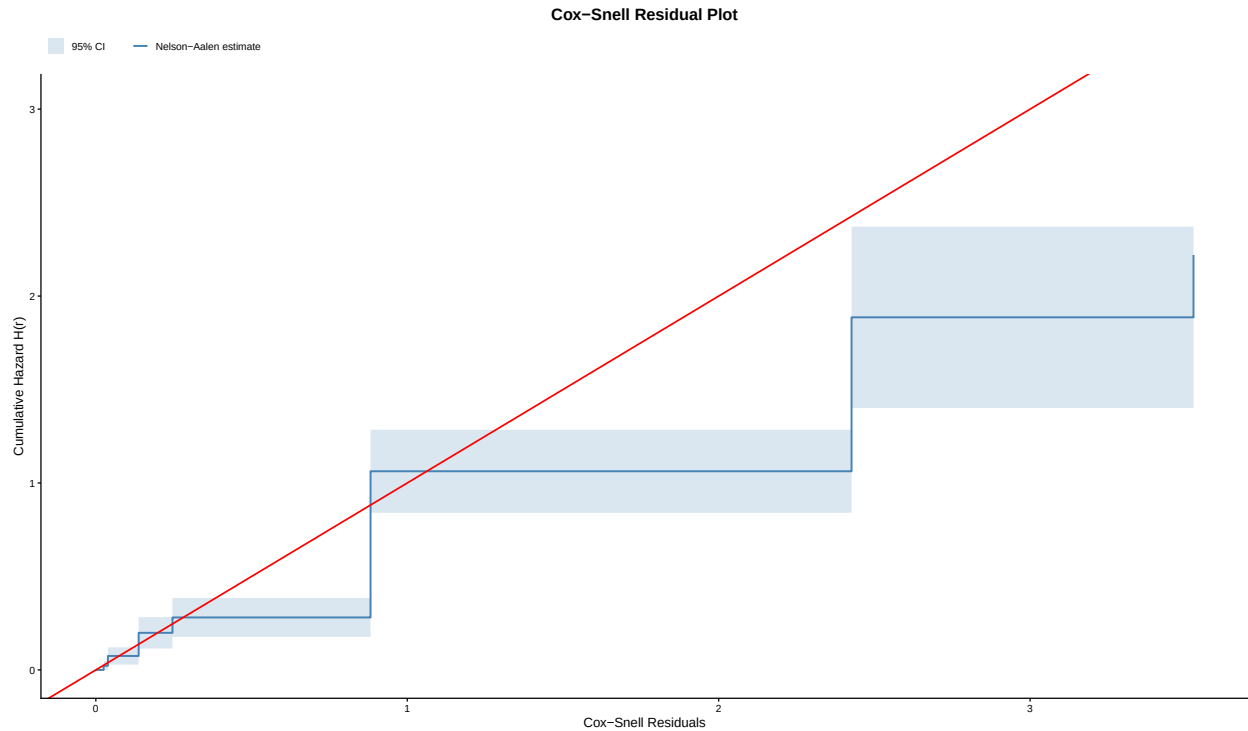
```
##          df      AIC
## tenta_F_4  3 811.4013
## tenta_F_2  1 815.4287
## tenta_F_3  2 817.3347
## tenta_F_1  1 911.8294
## tenta_F_0  0 916.5667
```

The model with tum_state_early has the lowest AIC and is therefore selected (tenta_F_2).

6.3 Goodness-of-fit evaluation

Assess goodness-of-fit of the selected model using Cox-Snell residuals.

```
plot_cox_snell_cox(tenta_F_2, tenta_df,
                    time_col = "age_followup", event_col = "tenta_event")
```



The curve follows a 45 degree exponential, we validate the fit of the selected model.

6.4 Selected model summary

```
summary(coxph(Surv(age_followup, tenta_event) ~ tum_state_early, data = tenta_df))
```

```
## Call:
## coxph(formula = Surv(age_followup, tenta_event) ~ tum_state_early,
##       data = tenta_df)
##
## n= 144, number of events= 105
##
##               coef exp(coef) se(coef)      z Pr(>|z|)
## tum_state_earlyTumoral  3.5653   35.3507   0.5975  5.967 2.42e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##               exp(coef) exp(-coef) lower .95 upper .95
## tum_state_earlyTumoral   35.35    0.02829    10.96    114
##
## Concordance= 0.734 (se = 0.026 )
## Likelihood ratio test= 103.1 on 1 df,  p=<2e-16
## Wald test               = 35.6 on 1 df,  p=2e-09
## Score (logrank) test = 76.73 on 1 df,  p=<2e-16
```

7 Median times by tumoral state

Estimate cumulative incidence functions for the supernumerary tentacles event, accounting for the competing risk of death, stratified by sleep tumoral state.

```
tenta_cuminc_tum <- cuminc(Surv(age_followup, status_competing) ~ tum_state_early, data =
tenta_df)
```

For each stratum, extract the median time to event and its 95CI.

```
tenta_medians <- tidy(tenta_cuminc_tum) %>%
  dplyr::filter(outcome == "1") %>%
  dplyr::group_by(strata) %>%
  dplyr::arrange(time) %>%
  dplyr::summarise(
    median_time = { idx <- which(estimate >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    ci_low = { idx <- which(conf.high >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    ci_high = { idx <- which(conf.low >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    .groups = "drop")

print(tenta_medians)
```

```
## # A tibble: 2 x 4
##   strata median_time ci_low ci_high
##   <fct>      <dbl> <dbl> <dbl>
## 1 Healthy      NA      NA      NA
## 2 Tumoral       4       4       4
```

8 Cumulative incidences by week and tumoral state

Convert the continuous cumulative incidence function into a weekly summary table: for each stratum of tumoral state and each week, show the estimate with its 95CI.

```
tenta_incidence <- tidy(tenta_cuminc_tum) %>%
  dplyr::filter(outcome == "1") %>%
  dplyr::mutate(week = ceiling(time)) %>%
  dplyr::group_by(strata, week) %>%
  dplyr::filter(time == max(time)) %>%
  dplyr::slice(1) %>%
  dplyr::ungroup() %>%
  dplyr::select(strata, week, estimate_cuminc = estimate, conf.low, conf.high) %>%
  dplyr::arrange(strata, week)

print(tenta_incidence)
```

```
## # A tibble: 14 x 5
##   strata week estimate_cuminc conf.low conf.high
```

```
##      <fct>    <dbl>          <dbl>    <dbl>    <dbl>
##  1 Healthy      0            0        NA        NA
##  2 Healthy      1            0        NA        NA
##  3 Healthy      2            0        NA        NA
##  4 Healthy      3            0        NA        NA
##  5 Healthy      4            0.075    0.0189    0.184
##  6 Healthy      5            0.075    0.0189    0.184
##  7 Healthy      6            0.075    0.0189    0.184
##  8 Tumoral      0            0        NA        NA
##  9 Tumoral      1            0.0673   0.0295    0.127
## 10 Tumoral      2            0.183    0.115     0.263
## 11 Tumoral      3            0.25     0.171     0.337
## 12 Tumoral      4            0.837    0.750     0.895
## 13 Tumoral      5            0.971    0.911     0.991
## 14 Tumoral      6            0.981    0.915     0.996
```

9 Cumulative proportions by week and tumoral state

Compute raw cumulative proportions of supernumerary tentacles occurrence per week, stratified by tumoral state.

```
tenta_proportions <- tenta_df %>%
  dplyr::mutate(week_tenta_event = ceiling(age_followup)) %>%
  dplyr::group_by(tum_state_early) %>%
  dplyr::summarise(total_N = n(),
    w1 = sum(week_tenta_event <= 1 & tenta_event == 1),
    w2 = sum(week_tenta_event <= 2 & tenta_event == 1),
    w3 = sum(week_tenta_event <= 3 & tenta_event == 1),
    w4 = sum(week_tenta_event <= 4 & tenta_event == 1),
    w5 = sum(week_tenta_event <= 5 & tenta_event == 1),
    w6 = sum(week_tenta_event <= 6 & tenta_event == 1),
    p1 = w1 / total_N,
    p2 = w2 / total_N,
    p3 = w3 / total_N,
    p4 = w4 / total_N,
    p5 = w5 / total_N,
    p6 = w6 / total_N,
    .groups = "drop") %>%
  tidyr::pivot_longer(cols = matches("^wp[1-6]$"),
    names_to = c(".value", "week"),
    names_pattern = "(w|p)([1-6])") %>%
  dplyr::mutate(week = as.numeric(week)) %>%
  dplyr::select(tum_state_early, week, n = w, total_N, raw_proportion = p)

print(tenta_proportions)
```

```
## # A tibble: 12 x 5
##   tum_state_early  week      n total_N raw_proportion
##   <fct>          <dbl> <int>   <int>         <dbl>
## 1 Healthy          1      0     40          0
## 2 Healthy          2      0     40          0
## 3 Healthy          3      0     40          0
```


## 4 Healthy	4	3	40	0.075
## 5 Healthy	5	3	40	0.075
## 6 Healthy	6	3	40	0.075
## 7 Tumoral	1	7	104	0.0673
## 8 Tumoral	2	19	104	0.183
## 9 Tumoral	3	26	104	0.25
## 10 Tumoral	4	87	104	0.837
## 11 Tumoral	5	101	104	0.971
## 12 Tumoral	6	102	104	0.981

10 Plots of cumulative incidence of supernumerary tentacles development by tumoral state

```
cuminc(Surv(age_followup, factor(status_competing)) ~ tum_state_early,
       data = tenta_df) %>%
  ggcuminc(outcome = "1") +
  add_confidence_interval() +
  labs(x = "Age (weeks)",
       y = "Cumulative incidence of\nsupernumerary tentacle development") +
  theme_bw() +
  theme(axis.text = element_text(color = "black", size = 14),
        axis.title.x = element_text(color = "black", size = 14, margin = margin(t = 10)),
        axis.title.y.right = element_text(color = "black", size = 14, angle = 90, margin = margin(l = 15)),
        plot.title = element_text(color = "black", size = 16, face = "bold", hjust = 0.5, margin = margin(b = 15)),
        plot.margin = margin(l = 15, r = 15, t = 15, b = 15),
        legend.position = "top",
        legend.justification = "center",
        legend.text = element_text(size = 13),
        legend.title = element_text(size = 14, face = "bold")) +
  scale_x_continuous(limits = c(0, 6),
                    breaks = seq(0, 6, by = 2)) +
  scale_y_continuous(limits = c(0, 1),
                    breaks = seq(0, 1, by = 0.25),
                    labels = percent,
                    position = "right") +
  scale_color_manual(values = tum_colors) +
  scale_fill_manual(values = tum_colors)
```

